# Embedded OS Implementation
# Spring 2025 Project #2

M11302149 電子碩一 趙孟哲

## [PART I ] EDF Scheduler Implementation [50%]

### A. Results of examples. (40%)

**Taskset 1:**

TaskSet.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢

```
1 0 2 6
2 0 5 9
```

Output.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明

```
2       Completion      task( 1)( 0)    task( 2)( 0)            2               0               4
7       Completion      task( 2)( 0)    task( 1)( 1)            7               2               2
 9      Completion      task( 1)( 1)    task( 2)( 1)            3               1               3
12      Preemption      task( 1)( 2)    task( 1)( 2)
14      Completion      task( 1)( 2)    task( 2)( 1)            2               0               4
16      Completion      task( 2)( 1)    task(63)                7               2               2
18      Preemption      task(63)        task( 1)( 3)
20      Completion      task( 1)( 3)    task( 2)( 2)            2               0               4
25      Completion      task( 2)( 2)    task( 1)( 4)            7               2               2
27      Completion      task( 1)( 4)    task( 2)( 3)            3               1               3
30      Preemption      task( 1)( 5)    task( 1)( 5)
32      Completion      task( 1)( 5)    task( 2)( 3)            2               0               4
34      Completion      task( 2)( 3)    task(63)                7               2               2
36      Preemption      task(63)        task( 1)( 6)
38      Completion      task( 1)( 6)    task( 2)( 4)            2               0               4
```

D:\RTOS\RTOS_M11302149_PA2_EDF\Microsoft\Windows\Kernel\OS2\VS\Debug\OS2.exe

```
OSTick    created, Thread ID 15220
Task[ 63] created, Thread ID 23196
The file 'TaskSet.txt' was opened
Task[  1] created, Thread ID  9076
Task[  2] created, Thread ID 22060
Insert(2, 0)
Insert(1, 0)
Tick 0: heap: (1,0) (2,0)
Tick 1: heap: (1,0) (2,0)
 2      Completion      task( 1)( 0)    task( 2)( 0)            2               0               4
Tick 2: heap: (2,0)
Tick 3: heap: (2,0)
Tick 4: heap: (2,0)
Tick 5: heap: (2,0)
Insert(1, 1)
Tick 6: heap: (2,0) (1,1)
 7      Completion      task( 2)( 0)    task( 1)( 1)            7               2               2
Tick 7: heap: (1,1)
Tick 8: heap: (1,1)
Insert(2, 1)
 9      Completion      task( 1)( 1)    task( 2)( 1)            3               1               3
Tick 9: heap: (2,1)
Tick 10: heap: (2,1)
Tick 11: heap: (2,1)
Insert(1, 2)
12      Preemption      task( 1)( 2)    task( 1)( 2)
Tick 12: heap: (1,2) (2,1)
Tick 13: heap: (1,2) (2,1)
14      Completion      task( 1)( 2)    task( 2)( 1)            2               0               4
Tick 14: heap: (2,1)
```

## Taskset 2:

TaskSet.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V)

```
1 0 2 4
2 0 4 7
```

Output.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明

```
2       Completion      task( 1)( 0)    task( 2)( 0)            2               0               2
6       Completion      task( 2)( 0)    task( 1)( 1)            6               2               1
8       Completion      task( 1)( 1)    task( 1)( 3)            4               2               0
10      Completion      task( 1)( 2)    task( 2)( 1)            2               0               2
14      Completion      task( 2)( 1)    task( 1)( 3)            7               3               0
16      Completion      task( 1)( 3)    task( 1)( 5)            4               2               0
18      Completion      task( 1)( 4)    task( 2)( 2)            2               0               2
21      MissDeadline    task( 2)( 2)    ------------------------
```

D:\RTOS\RTOS_M11302149_PA2_EDF\Microsoft\Windows\Kernel\OS2\VS\Debug\OS2.exe

```
 6      Completion      task( 2)( 0)    task( 1)( 1)            6               2               1
Tick 6: heap: (1,1)
Insert(2, 1)
Tick 7: heap: (1,1) (2,1)
Insert(1, 2)
 8      Completion      task( 1)( 1)    task( 1)( 3)            4               2               0
Tick 8: heap: (1,2) (2,1)
Tick 9: heap: (1,2) (2,1)
10      Completion      task( 1)( 2)    task( 2)( 1)            2               0               2
Tick 10: heap: (2,1)
Tick 11: heap: (2,1)
Insert(1, 3)
Tick 12: heap: (2,1) (1,3)
Tick 13: heap: (2,1) (1,3)
Insert(2, 2)
14      Completion      task( 2)( 1)    task( 1)( 3)            7               3               0
Tick 14: heap: (1,3) (2,2)
Tick 15: heap: (1,3) (2,2)
Insert(1, 4)
16      Completion      task( 1)( 3)    task( 1)( 5)            4               2               0
Tick 16: heap: (1,4) (2,2)
Tick 17: heap: (1,4) (2,2)
18      Completion      task( 1)( 4)    task( 2)( 2)            2               0               2
Tick 18: heap: (2,2)
Tick 19: heap: (2,2)
Insert(1, 5)
Tick 20: heap: (2,2) (1,5)
Insert(2, 2)
21      MissDeadline    task( 2)( 2)    ------------------------
```

# B. Description of implementation (10%)

我使用 **min-heap** 取代 ready table 來實作 EDF scheduler。採用 min-heap 取代 ready table 的理由在於能大幅降低 scheduler 演算法的時間複雜度：當有新 job 到達時，只要呼叫 heap 插入並自動調整（heapify-up），或在工作完成時呼叫刪除最小元素並調整（heapify-down），這兩個操作的時間複雜度都穩定在 O(log N)；而在排程決策階段，直接對堆頂做 peek 就能以 O(1) 取得高優先權的工作，無須掃描整個 heap；同時，由於 heap 在插入與刪除時就自動維持 deadline 排序，整體程式邏輯更為簡潔清晰，也更易於維護。上面程式執行結果可以看到 terminal 有 heap 的維護資訊。

本實作採用最早截止期限優先（EDF）演算法，並以最小堆管理所有準備好但尚未完成的工作。在 os_core.c 中，我將工作共分成四個模組實作：Consumer、Activer、Checker 與 EDF_Scheduler。取代系統原本的 scheduler。

- Consumer()：

  Consumer 會從 min-heap 頂端取出最早截止的節點（min_node），並將其剩餘執行時間（executetime）減一。若 executetime 減至 0，即代表該次工作已結束，此時透過任務控制區陣列 OSTCBPrioTbl 將對應任務的工作編號（TaskNumber）加一，為下一次週期性喚醒做準備。整個過程僅在堆 not empty、節點未被標示為 DONE 且 executetime 大於 0 時才會執行。

```
// Consumer: Decrements execution time of the heap's minimum node
void Consumer() {
    EDF_Node* min_node = EDF_HeapPeekMin();
    if (min_node != NULL && min_node->flag != DONE && min_node->executetime > 0) {
        min_node->executetime--;
        if (min_node->executetime == 0) {
            OSTCBPrioTbl[min_node->task_id]->TaskNumber++;
        }
    }
}
```

- Activer()：

  Activer 在每個 Tick interrpt 到達時，走訪所有使用者任務的 TCBList（OSTCBList），對每個非空閒優先級的任務檢查：(current_time – ArriveTime) % period == 0（週期任務的喚醒規則）。若條件成立，表示有新工作到達，於是配置並初始化一個新的 EDF_Node：設定任務編號（task_id）、工作序號（job_no）、到達時間（arrival）、絕對截止期限（arrival+period）、執行時間（execution_time）、旗標（flag）等，然後呼叫 EDF_HeapInsert 將它插入 heap，並以 new_job (Boolean) 標示本輪有新工作產生。

```
// Activer: Generates a new job for each task in OSTCBList if periodic condition is met
BOOLEAN Activer() {
    INT32U current_time = OSTime;
    BOOLEAN new_job = OS_FALSE; // Flag to indicate if any new job is generated

    // Traverse the OSTCBList
    OS_TCB* ptcb = OSTCBList;
    while (ptcb != NULL && ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO) {
        if ((current_time - ptcb->ArriveTime) % ptcb->period == 0) {
            // Generate a new job
            EDF_Node* new_node = (EDF_Node*)malloc(sizeof(EDF_Node));
            if (new_node == NULL) {
                // Memory allocation failure, continue to next task
                ptcb = ptcb->OSTCBNext;
                continue;
            }
            new_node->task_id = ptcb->TaskID;
            new_node->job_no = ptcb->TaskNumber;
            new_node->arrival = current_time;
            new_node->deadline = current_time + ptcb->period;
            new_node->executetime = ptcb->execution_time;
            new_node->flag = NORMAL;
            new_node->miss_reported = 0;
            printf("Insert(%u, %u)\n", new_node->task_id, new_node->job_no);
            EDF_HeapInsert(new_node);
            new_job = OS_TRUE; // Mark that a new job was generated
        }
        ptcb = ptcb->OSTCBNext; // Move to next task
    }

    return new_job; // Return 1 if any new job was generated, 0 otherwise
}
```

- Checker()：

  Checker 先由 EDF_HeapPeekMin 檢查堆頂節點：若該節點 executetime 已為 0 但尚未標示為 DONE，便將其 flag 設為 DONE，表示該工作已執行完畢。接著呼叫 EDF_check_miss 掃描整個 heap，看是否有因當前時間超過 deadline 而逾期的節點；若有，便回傳該 miss_node，供後續排程流程處理。

```
// Checker: Marks heap's minimum node as DONE if completed; checks for missed deadlines
EDF_Node* Checker() {
    // Check if the minimum node is done
    EDF_Node* min_node = EDF_HeapPeekMin();
    if (min_node != NULL && min_node->executetime == 0 && min_node->flag != DONE) {
        min_node->flag = DONE;
    }

    // Check all nodes for missed deadlines
    EDF_Node* miss_node = EDF_check_miss();
    return miss_node;
}
```

- EDF_Scheduler()：

  EDF_Scheduler 綜合上述三個模組的輸入，負責最終的任務切換與事件記錄。會設定 OSTCBHighRdy 和 OSPrioHighRdy，選中 EDF 中優先的 job。如果 heap 為空，則選中 IDLE_Task

```
void EDF_Scheduler(BOOLEAN new_job_in, EDF_Node* miss_node) {
    BOOLEAN done = OS_FALSE;
    EDF_Node* min_node = NULL;
    EDF_Node* min_node_copy = NULL;

    if (heap_size>0) {
        min_node = EDF_HeapPeekMin();
        if (min_node->flag == DONE) {
            done = OS_TRUE;
            min_node_copy = EDF_HeapExtractMin();
        }
    }
    if (miss_node) {
        PrintTask("MissDeadLine", NULL, miss_node);
        EDF_HeapClear();
        OSRunning = OS_FALSE;
        exit(1);
    }
    min_node = EDF_HeapPeekMin();
    if (new_job_in || done == OS_TRUE) {

        if (heap_size == 0) {
            OSTCBHighRdy = OSTCBPrioTbl[OS_TASK_IDLE_PRIO];
            OSPrioHighRdy = OS_TASK_IDLE_PRIO;
        }
        else {
            OSTCBHighRdy = OSTCBPrioTbl[min_node->task_id];
            OSPrioHighRdy = min_node->task_id;
        }


    }
```

```
    if (done == OS_TRUE) {
        PrintTask("Completion", min_node_copy, NULL);
        free(min_node_copy);
        //printf("SW to %u\n", OSPrioHighRdy);
    }
    else if (OSTCBHighRdy != OSTCBCur && OSTime != 0) {
        PrintTask("Preemption", min_node, NULL);
        //printf("SW to %u\n", OSPrioHighRdy);
    }
}
```

在 OSTimeTick 中，每次 tick interupt 到，依序呼叫 Consumer 處理最早截止作業的執行時間遞減，Checker 標記已完成並回傳 miss 節點，Activer 根據週期條件插入新作業並回傳 flag，EDF_Scheduler 根據新作業與 miss 結果切換任務並印出 Preemption／Completion 訊息，最後 EDF_print 顯示 heap 中資訊(debug 用)。

```
void  OSTimeTick(void)
{
```

```
    if (OSRunning == OS_TRUE) {
        /*setting the end time for the os*/
        if (OSTimeGet() > SYSTEM_END_TIME) {
            EDF_HeapClear();
            OSRunning = OS_FALSE;
            exit(0);
        }
        /*Setting the end time for the OS*/

        Consumer();
        EDF_Node* min_node = Checker();
        BOOLEAN new_job_in = Activer();
        EDF_Scheduler(new_job_in, min_node);
        EDF_print();
```

Os_Start 也一樣用我設計的 scheduler：

```
void  OSStart (void)
{
    if (OSRunning == OS_FALSE) {
        EDF_HeapInit();
        Consumer();
        BOOLEAN new_job_in = Activer();
        EDF_Scheduler(new_job_in, Checker());
        EDF_print();
        OSTCBCur = OSTCBHighRdy;
        OSStartHighRdy();
    }
}
```

在 OSIntExit 中，要把 OS_SchedNew() 和 OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy] 註解掉，即 timetick ISR 只使用 EDF_Scheduler()來設定 task 切換。

```
void  OSIntExit (void)
{
#if OS_CRITICAL_METHOD == 3u                                        /*
    OS_CPU_SR  cpu_sr = 0u;
#endif



    if (OSRunning == OS_TRUE) {
        OS_ENTER_CRITICAL();
        if (OSIntNesting > 0u) {                                    /*
            OSIntNesting--;
        }
        if (OSIntNesting == 0u) {                                   /*
            if (OSLockNesting == 0u) {                              /*
                //OS_SchedNew();
                //OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
                if (OSPrioHighRdy != OSPrioCur) {                   /*
```

以下是 heap 的實作邏輯 (在 os_core.c 中)：

1. swap 交換陣列中兩個元素。
2. heap_up 與 heap_down 分別執行向上或向下的 heapify 以維持順序
3. EDF_HeapInsert 將新節點插入堆中。
4. EDF_HeapExtractMin 移除並回傳 heap top 最小元素。
5. EDF_HeapPeekMin 僅回傳最小節點不移除。
6. EDF_check_miss 掃描 heap 中是否有逾期節點並回傳。
7. EDF_HeapInit 初始化 heap 結構，EDF_HeapClear 清空 heap。
8. EDF_print 列印目前 heap 中所有節點資訊。

```
/*heap*/
static void swap(int i, int j);
static void heap_up(int i);
static void heap_down(int i);
static void EDF_HeapInsert(EDF_Node* node);
static EDF_Node* EDF_HeapExtractMin();
static EDF_Node* EDF_HeapPeekMin();
static EDF_Node* EDF_check_miss();
static void EDF_HeapInit();
static void EDF_HeapClear();
static void EDF_print();
/*heap*/
```

```
void swap(int i, int j) {
    EDF_Node* temp = edf_heap[i];
    edf_heap[i] = edf_heap[j];
    edf_heap[j] = temp;
}
```

heap_up 的邏輯：

從索引 i 開始，不斷比較這個節點和它的父節點。如果當前節點的 deadline 比父節點小，或 deadline 相同但 task_id 較小，就跟父節點交換，然後繼續往上迭代，否則停止。先比 deadline，若相同再比 task_id。

```
void heap_up(int i) {
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (edf_heap[i]->deadline < edf_heap[parent]->deadline ||
            (edf_heap[i]->deadline == edf_heap[parent]->deadline &&
                edf_heap[i]->task_id < edf_heap[parent]->task_id)) {
            swap(i, parent);
            i = parent;
        }
        else {
            break;
        }
    }
}
```

heap_down 的邏輯：

從索引 i 開始，不斷往下比較左右子節點。看左子節點（2i+1）和右子節點（2i+2）哪個最小，最小判準同樣是先比 deadline，再比 task_id，如果有子節點比自己更小，就跟子節點交換，並把 i 更新到子節點索引，然後繼續迭

代，否則停止。先比 deadline，若相同再比 task_id。

```c
void heap_down(int i) {
    int left, right, smallest;

    while (1) {
        left = 2 * i + 1;
        right = 2 * i + 2;
        smallest = i;

        if(left < heap_size &&
            (edf_heap[left]->deadline < edf_heap[smallest]->deadline ||
                (edf_heap[left]->deadline == edf_heap[smallest]->deadline &&
                    edf_heap[left]->task_id < edf_heap[smallest]->task_id))) {
            smallest = left;
        }
        if (right < heap_size &&
            (edf_heap[right]->deadline < edf_heap[smallest]->deadline ||
                (edf_heap[right]->deadline == edf_heap[smallest]->deadline &&
                    edf_heap[right]->task_id < edf_heap[smallest]->task_id))) {
            smallest = right;
        }
        if (smallest != i) {
            swap(i, smallest);
            i = smallest;
        }
        else {
            break;
        }
    }
}
```

```c
void EDF_HeapInsert(EDF_Node* node) {
    if (heap_size >= MAX_HEAP_SIZE) return;
    edf_heap[heap_size] = node;
    heap_up(heap_size);
    heap_size++;
}

EDF_Node* EDF_HeapExtractMin() {
    if (heap_size == 0) return NULL;
    EDF_Node* min_node = edf_heap[0];
    edf_heap[0] = edf_heap[--heap_size];
    heap_down(0);
    return min_node;
}

EDF_Node* EDF_HeapPeekMin() {
    if (heap_size == 0) return NULL;
    return edf_heap[0];
}
```

```
EDF_Node* EDF_check_miss() {
    EDF_Node* miss_node = NULL;
    for (int i = 0; i < heap_size; i++) {
        EDF_Node* node = edf_heap[i];
        if (node->flag != DONE && OSTime >= node->deadline) {
            node->flag = MISS;
            miss_node = node;
        }
        else if (node->flag != DONE && node->flag != MISS) {
            node->flag = NORMAL;
        }
    }
    return miss_node;
}

void EDF_print() {
    printf("Tick %u: heap: ", OSTime);
    if (heap_size == 0) {
        printf("HEAP Empty\n");
        return;
    }

    for (int i = 0; i < heap_size; i++) {
        EDF_Node* node = edf_heap[i];
        printf("(%u,%u) ",node->task_id,node->job_no);
    }
    printf("\n");
}
```

```
void EDF_HeapInit() {
    heap_size = 0;
}

void EDF_HeapClear() {
    // Iterate through all nodes in the heap
    for (int i = 0; i < heap_size; i++) {
        if (edf_heap[i] != NULL) {
            free(edf_heap[i]); // Free the dynamically allocated EDF_Node
            edf_heap[i] = NULL; // Prevent dangling pointer
        }
    }
    heap_size = 0; // Reset heap size
}
```

以下為 EDF_node 和 heap 的資料結構 (ucosii.h)：

```
/*task node heap*/
typedef enum { NORMAL, DONE, MISS } JobFlag;
typedef struct EDF_Node{
    INT8U task_id;
    INT8U job_no;
    INT32U arrival;
    INT32U deadline;
    INT32U executetime;
    JobFlag flag;
    INT8U miss_reported;
} EDF_Node;

EDF_Node* edf_heap[MAX_HEAP_SIZE];
int heap_size;
/*task node heap*/
```

os_cfg.h 指定 heap 最大數量：

```
#define MAX_HEAP_SIZE 16
```

Task 只跑無窮迴圈，只在 Timetick 時才做任務切換。

```
void task(void* p_arg) {
    task_para_set* task_data = (task_para_set*)p_arg;
    while (1) {
        OS_Dummy();

    }
```

# [ PART II ] CUS Scheduler Implementation [50%]

## A. Results of examples. (40%)

### Taskset 1:

TaskSet.txt - 記事本

檔案(F) 編輯(E) 格式(O)

```
1 0 2 8
2 0 3 10
3 0 4 15
4 25
```

AperiodicJobs.tx

檔案(F) 編輯(E) 格

```
0 12 3 25
1 14 2 33
```

目前我的 aperiodic job 的 response time 是從 arrival time 開始算而不是算 server 開始執行的時間。

Output.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明

```
2      Completion      task( 1)( 0)    task( 2)( 0)          2             0             6
 5     Completion      task( 2)( 0)    task( 3)( 0)          5             2             5
 9     Completion      task( 3)( 0)    task( 1)( 1)          9             5             6
11     Completion      task( 1)( 1)    task( 2)( 1)          3             1             5
12     Aperiodic job ( 0) arrives and sets CUS server' s deadline as 24.
14     Aperiodic job ( 1) arrives. Do nothing.
14     Completion      task( 2)( 1)    task( 4)( 0)          4             1             6
16     Preemption      task( 4)( 0)    task( 1)( 2)
18     Completion      task( 1)( 2)    task( 4)( 0)          2             0             6
19     Aperiodic job ( 0) is finished.
19     Completion      task( 4)( 0)    task( 3)( 1)          7             4           N/A
20     Preemption      task( 3)( 1)    task( 2)( 2)
23     Completion      task( 2)( 2)    task( 3)( 1)          3             0             7
24     Aperiodic job ( 1) sets CUS server' s deadline as  32.
26     Completion      task( 3)( 1)    task( 1)( 3)         11             7             4
28     Completion      task( 1)( 3)    task( 4)( 1)          4             2             4
30     Aperiodic job ( 1) is finished.
30     Completion      task( 4)( 1)    task( 2)( 3)         16            14           N/A
32     Preemption      task( 2)( 3)    task( 1)( 4)
34     Completion      task( 1)( 4)    task( 2)( 3)          2             0             6
35     Completion      task( 2)( 3)    task( 3)( 2)          5             2             5
39     Completion      task( 3)( 2)    task(63)              9             5             6
40     Preemption      task(63)        task( 1)( 5)
```

### Taskset 2:

TaskSet.txt - 記事本

檔案(F) 編輯(E) 格式(O)

```
1 0 2 4
2 0 3 9
3 30
```

AperiodicJobs.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢

```
0 1 3 24
1 11 2 39
```

Output.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明

```
1       Aperiodic job ( 0) arrives and sets CUS server' s deadline as 11.
2       Completion      task( 1)( 0)    task( 2)( 0)          2             0             2
4       Preemption      task( 2)( 0)    task( 1)( 1)
6       Completion      task( 1)( 1)    task( 2)( 0)          2             0             2
7       Completion      task( 2)( 0)    task( 3)( 0)          7             4             2
10      Aperiodic job ( 0) is finished.
10      Completion      task( 3)( 0)    task( 1)( 2)          9             6           N/A
11      Aperiodic job ( 1) arrives and sets CUS server' s deadline as 17.
12      Completion      task( 1)( 2)    task( 1)( 3)          4             2             0
14      Completion      task( 1)( 3)    task( 3)( 1)          2             0             2
16      Aperiodic job ( 1) is finished.
16      Completion      task( 3)( 1)    task( 2)( 1)          5             3           N/A
18      MissDeadline    task( 2)( 1)    ------------------------
```

## Taskset 3:

TaskSet.txt - 記事本

檔案(F) 編輯(E) 格式(O

```
1 0 3 6
2 0 4 10
3 10
```

AperiodicJobs.txt -

檔案(F) 編輯(E) 格式(

```
0 7 1 23
1 10 2 15
```

Output.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明

```
3       Completion      task( 1)( 0)    task( 2)( 0)          3             0             3
7       Aperiodic job ( 0) arrives and sets CUS server' s deadline as 17.
7       Completion      task( 2)( 0)    task( 1)( 1)          7             3             3
10      Aperiodic job( 1) arrives. But can't Scheduled. (miss absolute deadline) .
10      Completion      task( 1)( 1)    task( 3)( 0)          4             1             2
11      Aperiodic job ( 1) is finished.
11      Completion      task( 3)( 0)    task( 2)( 1)          4             3           N/A
12      Preemption      task( 2)( 1)    task( 1)( 2)
15      Completion      task( 1)( 2)    task( 2)( 1)          3             0             3
18      Completion      task( 2)( 1)    task( 1)( 3)          8             4             2
21      Completion      task( 1)( 3)    task( 2)( 2)          3             0             3
24      Preemption      task( 2)( 2)    task( 1)( 4)
27      Completion      task( 1)( 4)    task( 2)( 2)          3             0             3
28      Completion      task( 2)( 2)    task(63)              8             4             2
30      Preemption      task(63)        task( 1)( 5)
33      Completion      task( 1)( 5)    task( 2)( 3)          3             0             3
37      Completion      task( 2)( 3)    task( 1)( 6)          7             3             3
40      Completion      task( 1)( 6)    task( 2)( 4)          4             1             2
```

# B. Description of implementation (10%)

在 ucosii.h 新增資料結構以支援 CUS server。

```
//for server
OS_EVENT* serverQ;
void* serverQBuffer[APERIODIC_QUEUE_SIZE];
```

```c
/*Aperiodic Task structure*/
typedef struct aperiod_Param {
    INT16U TaskID;
    INT16U TaskArriveTime;
    INT16U TaskExecutionTime;
    INT16U TaskDeadLine;
} aperiod_Param;
aperiod_Param AperiodList[OS_MAX_SERVER];
int Aperiod_NUMBER;
/*Aperiodic Task structure*/

/*Server parameter*/
typedef struct sever_para {
    INT16U TaskID_server;
    INT16U Deadline;
    INT16U Budget;
    INT16U Size;
} sever_para;
sever_para Server_Para;
int server_cur;

/*Server parameter*/
```

在 os_cfg.h 中，設定常數設定：

```c
#define OS_MAX_SERVER 16
#define APERIODIC_QUEUE_SIZE  10
```

接著，基於 EDF schedule，在 os_core.c 修改 scheduler 邏輯：
Activer 新增 Aperiodic job 的處理邏輯，這邊使用 ServerQ (Os_Q) 來模擬 CUS 的 queue，當有 job 進入時將 Aperiodic job 放入 queue 中。然後使用 replensher。因為 Aperiodic txt 檔已經將 job 排序，只需要設定 server_cur 來維護目前 job 的 ariival 即可。

```c
// Traverse the OSTCBList
OS_TCB* ptcb = OSTCBList;
while (ptcb != NULL && ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO) {
    if (ptcb->period == 0) { //server
        if (AperiodList[server_cur].TaskArriveTime == current_time)
        {
            INT8U perr;
            aperiod_Param* job = (aperiod_Param*)OSQPeekTail(serverQ, &perr);  // Peek queue
            if (OSQPost(serverQ, &AperiodList[server_cur]) != OS_ERR_NONE) {
                printf("Q full. Dropping job!\n");
            }
            else {
                printf("Q Posted\n");
            }
            server_replenisher(OS_FALSE);

            server_cur++;
        }
    }
```

Server_replensher()：

管理一個 Constant Utilization Server（CUS）budget 的補充與排程。它先取得目前的系統時刻和 server 佇列狀態。

當參數 type 為 OS_FALSE 時表示有新工作到達，它會檢查此工作是否錯過 deadline，若錯過就丟棄並記錄；否則 queue 空，則計算並設定新的 server deadline 與 budget，接著將該工作以 Earliest Deadline First（EDF）方式插入 heap (注意：server 一次只能執行一個 aperiodic task，且 task ID 為 server ID)。

當參數 type 為 OS_TRUE 時表示伺服器到達截止時間，若此時仍有積壓工作，就再次補充預算並插入下一個工作。這個函式負責檢查 Aperiodic job 的

條件、根據演算法補充 budget (即 server 的 execute_time)、印出訊息與新增 job 節點到 heap 中。如以下邏輯：

*Replenishment Rules of a Constant Utilization Server of Size $\tilde{u}_s$*

R1 Initially, $e_s = 0$, and $d = 0$.

R2 When an aperiodic job with execution time $e$ arrives at time $t$ to an empty aperiodic job queue,

(a) if $t < d$, do nothing;

(b) if $t \geq d$, $d = t + e/\tilde{u}_s$, and $e_s = e$.

R3 At the deadline $d$ of the server,

(a) if the server is backlogged, set the server deadline to $d + e/\tilde{u}_s$ and $e_s = e$;

(b) if the server is idle, do nothing.

**[Spuri 96]**

- 當有非週期性工作到達時，先比較它的到達時間和目前伺服器的截止時間如果到達時間小於伺服器截止時間，就印出“Aperiodic job (工作編號) arrives. Do nothing.”表示伺服器忙碌但不做任何動作
- 然後如果此時伺服器剛好只有這一筆工作並且已經到了補充點，就計算新的截止時間並印出“Aperiodic job (工作編號) sets CUS server’s deadline as (新截止時間).”
- 否則如果到達時間大於等於伺服器的截止時間，就直接印出“Aperiodic job (工作編號) arrives and sets CUS server’s deadline as (新截止時間).”
- 接著檢查這筆工作的絕對截止時間是否已經比伺服器能服務的最晚時間還早，如果是，就印出“Aperiodic job (工作編號) arrives. But can’t Scheduled. (miss absolute deadline)”表示該工作無法在自己期限內完成
- 工作執行完畢時再印出“Aperiodic job (工作編號) is finished.”以標示它結束。

```
void server_replenisher(BOOLEAN type) {
    INT32U now = OSTimeGet();
    INT8U perr;
    aperiod_Param* job = (aperiod_Param*)OSQPeekTail(serverQ, &perr);  // Peek queue
    int backlogged = (job != NULL && perr == OS_ERR_NONE);

    if ((Output_err = fopen_s(&Output_fp, "./Output.txt", "a")) != 0) {
        printf("Error open Output.txt!\n");
    }

    if (type == OS_FALSE && job!=NULL) {
        //arrival
        OS_Q* q = serverQ->OSEventPtr;
        if (AperiodList[server_cur].TaskDeadLine <
            (INT16U)(job->TaskArriveTime + (job->TaskExecutionTime * 100 / Server_Para.Size))) {
            printf("%2u\tAperiodic job(%2u) arrives. But can't Scheduled. (miss absolute deadline) .\n",
                now, AperiodList[server_cur].TaskID);
            fprintf(Output_fp, "%2u\tAperiodic job(%2u) arrives. But can't Scheduled. (miss absolute deadline) .\n",
                now, AperiodList[server_cur].TaskID);
            OSQAccept(serverQ, &perr);
        }
```

```
else if (now >= Server_Para.Deadline && q->OSQEntries == 1) {
    INT32U e = job->TaskExecutionTime;

    Server_Para.Deadline = (INT16U)(job->TaskArriveTime + (e * 100 / Server_Para.Size));
    Server_Para.Budget = e;


    printf("%2u\tAperiodic job (%2u) arrives and sets CUS server' s deadline as %2u. \n",
        now,job->TaskID, Server_Para.Deadline);
    fprintf(Output_fp,"%2u\tAperiodic job (%2u) arrives and sets CUS server' s deadline as %2u. \n",
        now,job->TaskID, Server_Para.Deadline);
    INT8U err;
    aperiod_Param* job = (aperiod_Param*)OSQPeekHead(serverQ, &err);
    EDF_Node* node = (EDF_Node*)malloc(sizeof(EDF_Node));
    if (node) {
        node->task_id = Server_Para.TaskID_server;
        node->job_no = job->TaskID;
        node->arrival = now;
        node->deadline = Server_Para.Deadline;
        node->executetime = job->TaskExecutionTime;
        node->flag = NORMAL;
        node->miss_reported = 0;
        EDF_HeapInsert(node);
        printf("Inssert job (%2u) to heap", job->TaskID);
    }
```

```
    else {
        printf("%2u\tAperiodic job (%2u) arrives. Do nothing.\n", now, job->TaskID);
        fprintf(Output_fp,"%2u\tAperiodic job (%2u) arrives. Do nothing.\n", now, job->TaskID);
    }
}
else {
    // deadline
    if (now == Server_Para.Deadline &&backlogged) {
        INT32U e = job->TaskExecutionTime;

        Server_Para.Deadline = (INT16U)(Server_Para.Deadline + (e * 100 / Server_Para.Size));
        Server_Para.Budget = e;

        printf("%2u\tAperiodic job (%2u) sets CUS server' s deadline as %3u.\n",
            now, job->TaskID, Server_Para.Deadline);
        fprintf(Output_fp,"%2u\tAperiodic job (%2u) sets CUS server' s deadline as %3u.\n",
            now, job->TaskID, Server_Para.Deadline);

        INT8U err;
        aperiod_Param* job = (aperiod_Param*)OSQPeekHead(serverQ, &err);
        EDF_Node* node = (EDF_Node*)malloc(sizeof(EDF_Node));
        if (node) {
            node->task_id = Server_Para.TaskID_server;
            node->job_no = job->TaskID;
            node->arrival = now;
            node->deadline = Server_Para.Deadline;
            node->executetime = job->TaskExecutionTime;
            node->flag = NORMAL;
            node->miss_reported = 0;
            EDF_HeapInsert(node);
            printf("Inssert job (%2u) to heap", job->TaskID);
        }
    }
}
fclose(Output_fp);
```

Scheduler 如果 aperiodic job 做完要 dequeue。

```
void EDF_Scheduler(BOOLEAN new_job_in, EDF_Node* miss_node) {
    BOOLEAN done = OS_FALSE;
    EDF_Node* min_node = NULL;
    EDF_Node* min_node_copy = NULL;

    if (heap_size>0) {
        min_node = EDF_HeapPeekMin();
        if (min_node->flag == DONE) {
            done = OS_TRUE;
            min_node_copy = EDF_HeapExtractMin();
            if (min_node->task_id == Server_Para.TaskID_server)
            {



                INT8U err;
                aperiod_Param* msg  = OSQAccept(serverQ, &err); // deQ
                printf("%2u\tAperiodic job (%2u) is finished.\n", OSTime, msg->TaskID);
                if ((Output_err = fopen_s(&Output_fp, "./Output.txt", "a")) != 0) {
                    printf("Error open Output.txt!\n");
                }
                fprintf(Output_fp, "%2u\tAperiodic job (%2u) is finished.\n", OSTime, msg->TaskID);
                fclose(Output_fp);
            }
        }
    }
```

在 Timetick 中使用 server_replenisher 在每個 tick 都檢查 server 的 deadline，即 R3 replenish 邏輯。

```
Consumer();

EDF_Node* min_node = Checker();
BOOLEAN new_job_in = Activer();
server_replenisher(OS_TRUE);
EDF_Scheduler(new_job_in, min_node);
EDF_print();
```

另外，原生 OS_Q 沒有 peek 功能，我實作兩個 fuction 能夠 peek queue 頭和尾。

```
aperiod_Param* OSQPeekHead(OS_EVENT* pevent, INT8U* perr) {
    aperiod_Param* msg = NULL;
    OS_Q* q;
    if (perr == NULL) return NULL;
    OS_ENTER_CRITICAL();
    if (pevent->OSEventType != OS_EVENT_TYPE_Q) {
        *perr = OS_ERR_EVENT_TYPE;
    }
    else {
        q = (OS_Q*)pevent->OSEventPtr;
        if (q->OSQEntries == 0) {
            *perr = OS_ERR_Q_EMPTY;
        }
        else {
            msg = *((aperiod_Param**)q->OSQOut);
            *perr = OS_ERR_NONE;
        }
    }
    OS_EXIT_CRITICAL();
    return msg;
}
```

```
aperiod_Param* OSQPeekTail(OS_EVENT* pevent, INT8U* perr) {
    aperiod_Param* msg = NULL;
    OS_Q* q;
    void** last_ptr;
    if (perr == NULL) return NULL;
    OS_ENTER_CRITICAL();
    if (pevent->OSEventType != OS_EVENT_TYPE_Q) {
        *perr = OS_ERR_EVENT_TYPE;
    }
    else {
        q = (OS_Q*)pevent->OSEventPtr;
        if (q->OSQEntries == 0) {
            *perr = OS_ERR_Q_EMPTY;
        }
        else {
            last_ptr = q->OSQIn;
            if (--last_ptr < q->OSQStart) {
                last_ptr = q->OSQEnd;
            }
            msg = *((aperiod_Param**)last_ptr);
            *perr = OS_ERR_NONE;
        }
    }
    OS_EXIT_CRITICAL();
    return msg;
}
```

在 app_hook.c 中，讀取 txt 檔時要改成一行一行讀，如果 4 欄資料為週期任務，2 欄則為 server 的參數。

```
while (fgets(str, sizeof(str), fp)) {
    // blank row ,skip
    if (str[0] == '\n' || str[0] == '\0') continue;

    count = 0;
    ptr = strtok_s(str, " \t\r\n", &pTmp);
    while (ptr != NULL && count < 4) {
        values[count++] = atoi(ptr);
        ptr = strtok_s(NULL, " \t\r\n", &pTmp);
    }

    if (count == 4) {
        // Periodic task
        TaskParameter[TASK_NUMBER].TaskID = values[0];
        TaskParameter[TASK_NUMBER].TaskPriority = values[0];
        TaskParameter[TASK_NUMBER].TaskArriveTime = values[1];
        TaskParameter[TASK_NUMBER].TaskExecutionTime = values[2];
        TaskParameter[TASK_NUMBER].TaskPeriodic = values[3];
        p2id[values[0]] = values[0];
        TASK_NUMBER++;
    }
    else if (count == 2) {
        // Server
        Server_Para.TaskID_server= values[0];
        TaskParameter[TASK_NUMBER].TaskID = values[0];
        TaskParameter[TASK_NUMBER].TaskPriority = values[0];
        TaskParameter[TASK_NUMBER].TaskArriveTime = 0;
        TaskParameter[TASK_NUMBER].TaskExecutionTime = 0;
        TaskParameter[TASK_NUMBER].TaskPeriodic = 0; // aperiodic
        server_cur = 0;
        Server_Para.Size = values[1];
        Server_Para.Deadline = 0;
        TASK_NUMBER++;
    }
}
fclose(fp);
```

讀取 AperiodicJobs.txt：

```
// aperiodic jobs
if ((err = fopen_s(&fp, "AperiodicJobs.txt", "r")) != 0) {
    printf("The file 'AperiodicJobs.txt' was not opened\n");
    return;
}

printf("The file 'AperiodicJobs.txt' was opened\n");

while (fgets(str, sizeof(str), fp)) {
    if (str[0] == '\n' || str[0] == '\0') continue;

    count = 0;
    ptr = strtok_s(str, " \t\r\n", &pTmp);
    while (ptr != NULL && count < 4) {
        values[count++] = atoi(ptr);
        ptr = strtok_s(NULL, " \t\r\n", &pTmp);
    }

    if (count == 4) {
        AperiodList[Aperiod_NUMBER].TaskID = values[0];
        AperiodList[Aperiod_NUMBER].TaskArriveTime = values[1];
        AperiodList[Aperiod_NUMBER].TaskExecutionTime = values[2];
        AperiodList[Aperiod_NUMBER].TaskDeadLine = values[3];
        Aperiod_NUMBER++;
    }
}

fclose(fp);
```

# [ BOUNS] Button-triggered Aperiodic Job

實驗課時，有將按鈕觸發 arduino aperiodic jos 的功能寫出。